The complete ER / EER atlas

DATABASE DESIGN · ENTITY DICTIONARY · VIVA COMPANION

CampusOS

Every entity. Every attribute.

Every relationship, explained.

16 implemented tables | **15 foreign keys** | **Chen + relational views** | **Proposed EER extensions**

A detailed, landscape-format reference for the college management system. Read the implemented schema first, then explore composite, multivalued and derived attributes, inheritance and relational mapping.

> **Honest scope.** The entity dictionary is generated from `web/lib/schema.mjs` and portal DDL in `web/lib/portal.mjs`, using an isolated in-memory SQLite database. The Next.js/SQLite build is independent of the original PostgreSQL/FastAPI build. Person, multi-phone relations and ISA subtypes are **proposed teaching models**, not implemented tables.

Reading map & design boundaries

01 / Implemented system

- Three readable relationship plates: academic structure; teaching and assessment; library and authentication.
- A complete FK/cardinality ledger with minimum and maximum participation.
- Sixteen entity sheets with all stored columns, types, nullability, primary/foreign/unique keys, defaults and business constraints.

02 / Conceptual EER

- Chen notation, composite names, multivalued phones and derived age.
- Person → Student / Faculty: proposed ISA hierarchy, inheritance, specialization and generalization.
- Disjointness, overlapping membership, total and partial completeness.
- Weak entities, associative entities, mapping rules and viva answers.

What the diagrams mean

In the relationship plates, arrows run from **referenced parent → referencing child**. Labels give the number of children allowed for one parent. These overview plates use boxes for readability; entity sheets use Chen rectangles and attribute ellipses.

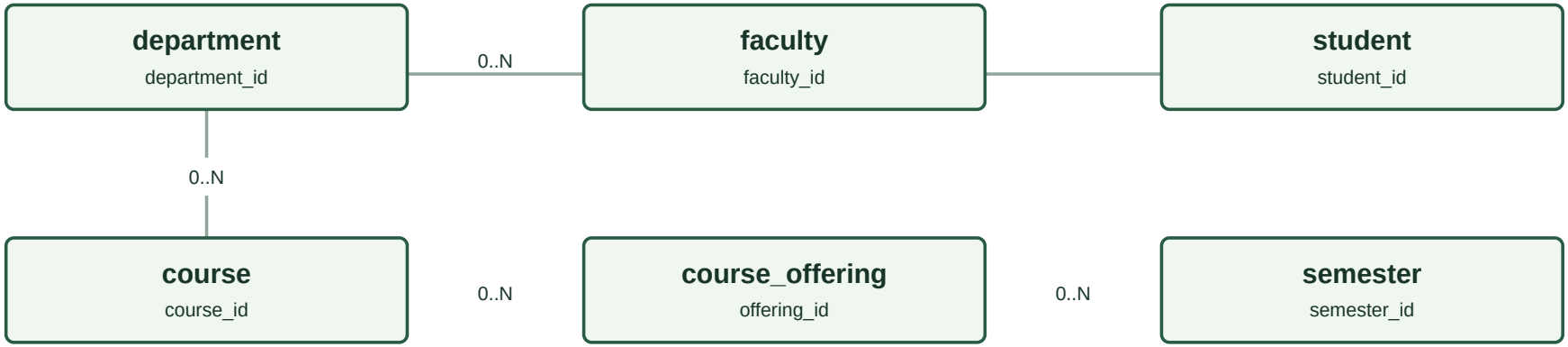
The complete relationship ledger is authoritative for both directions. A NOT NULL FK means each child has exactly one parent; a nullable FK permits zero or one. Unless a unique FK restricts it, a parent may have zero to many children.

> A foreign key does not require a parent to have children. A department can have no courses; an enrollment can have no grade. Cardinality is not a substitute for capacity or availability checks.

Key legend

PK primary key · **FK** foreign key · **UQ** unique constraint · **NN** required / NOT NULL. Underlined Chen attributes are PKs. Composite UQs are listed separately, never as individually unique columns.

Academic structure

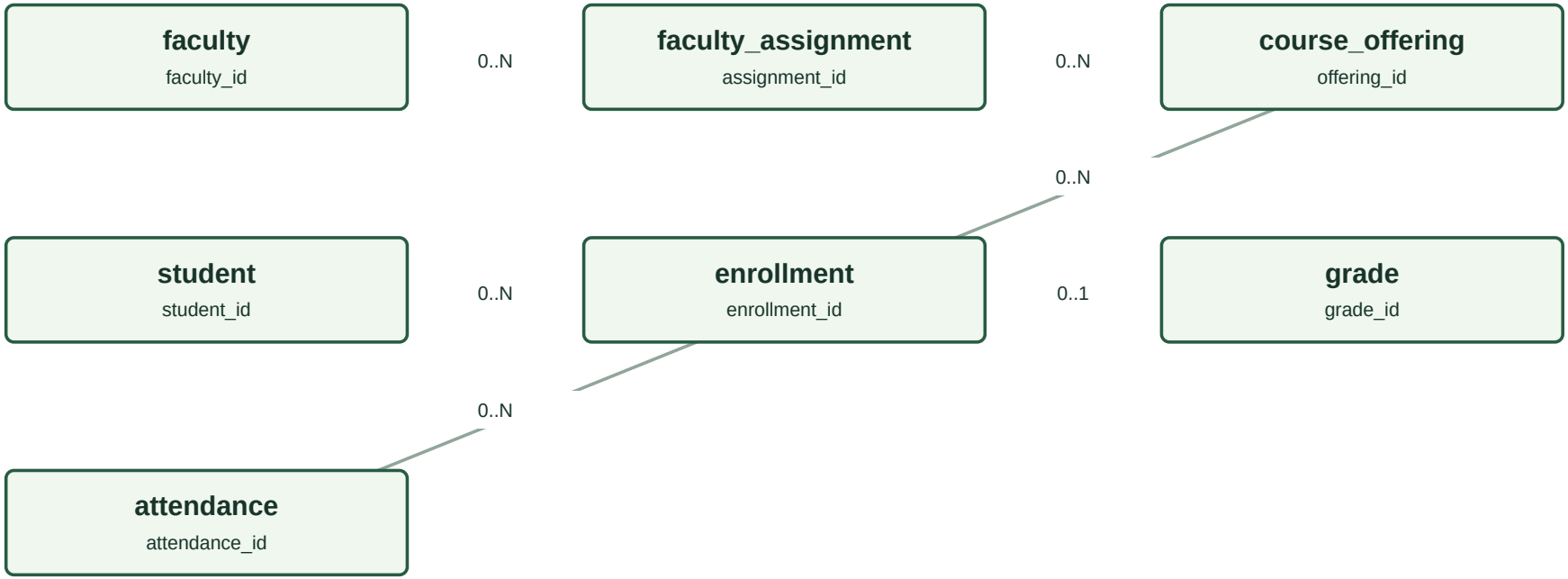


Department → Student: each student has 0..1 department because the FK is nullable. Each faculty member and course must belong to exactly one department.

Course vs Offering: the catalog course is reusable; a course offering fixes the course, semester and section. UNIQUE(course_id, semester_id, section) prevents a duplicate scheduled section.

> These are associations, not inheritance: a CourseOffering is not an ISA subtype of Course. It represents a scheduled instance and has its own capacity and identity.

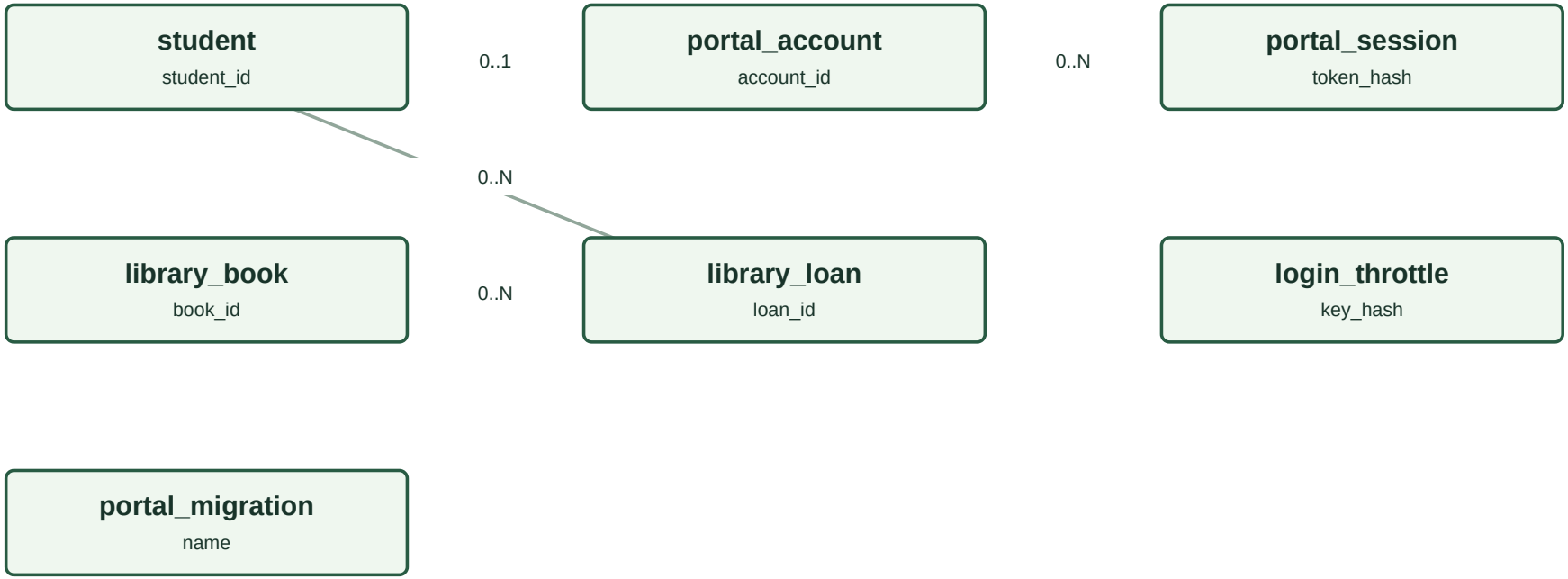
Teaching, registration & assessment



Two M:N relationships: Faculty ↔ Offering is resolved by faculty_assignment; Student ↔ Offering is resolved by enrollment. Their pairwise UNIQUE constraints prevent duplicate links.

Assessment: grade.enrollment_id is mandatory and unique, so each grade has one enrollment and an enrollment has at most one grade. Attendance is unique per enrollment and date.

Library, accounts & operational state



Student ↔ Book: a loan is a historical borrowing event. The same student can borrow the same title repeatedly; there is no UNIQUE(student_id, book_id) constraint.

Accounts: a student account references exactly one student; an admin references none. login_throttle and portal_migration are deliberately unconnected operational tables, not missing relationships.

> library_book stores a title and a copies count, not individual physical-copy identities. No fine, reservation or multi-author child table is implemented.

Complete relationship & participation ledger

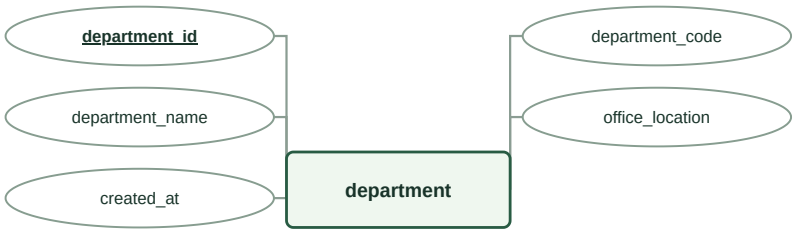
Read “parent → child” with the two participation columns. All 15 foreign keys in the implemented extended schema are included.

Referenced parent → child	Child foreign key	Children / parent	Parents / child	ON DELETE / UPDATE
department → student	`department_id`	0..N	0..1	RESTRICT / CASCADE
department → faculty	`department_id`	0..N	1..1	RESTRICT / CASCADE
department → course	`department_id`	0..N	1..1	RESTRICT / CASCADE
semester → course_offering	`semester_id`	0..N	1..1	RESTRICT / CASCADE
course → course_offering	`course_id`	0..N	1..1	RESTRICT / CASCADE
course_offering → faculty_assignment	`offering_id`	0..N	1..1	RESTRICT / CASCADE
faculty → faculty_assignment	`faculty_id`	0..N	1..1	RESTRICT / CASCADE
course_offering → enrollment	`offering_id`	0..N	1..1	RESTRICT / CASCADE
student → enrollment	`student_id`	0..N	1..1	RESTRICT / CASCADE
enrollment → grade	`enrollment_id`	0..1	1..1	RESTRICT / CASCADE
student → portal_account	`student_id`	0..1	0..1	RESTRICT / NO ACTION
portal_account → portal_session	`account_id`	0..N	1..1	CASCADE / NO ACTION
enrollment → attendance	`enrollment_id`	0..N	1..1	RESTRICT / NO ACTION
library_book → library_loan	`book_id`	0..N	1..1	RESTRICT / NO ACTION
student → library_loan	`student_id`	0..N	1..1	RESTRICT / NO ACTION

portal_account.student_id is conditionally required by role even though its column is nullable. All domain deletion actions are RESTRICT; deleting a portal_account cascades its portal_session rows. “NO ACTION” shown for some UPDATE actions is SQLite's default, not ON UPDATE CASCADE.

department

Organizational owner of academic records.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> Department code and name are separately unique. The code must contain 2–10 uppercase letters.

Attribute	SQL type	Keys	NN	Default
department_id	INTEGER	PK	Yes	—
department_code	VARCHAR(10)	UQ	Yes	—
department_name	VARCHAR(100)	UQ	Yes	—
office_location	VARCHAR(100)	—	No	—
created_at	TIMESTAMP	—	Yes	CURRENT_TIMESTAMP

Identity & alternate keys

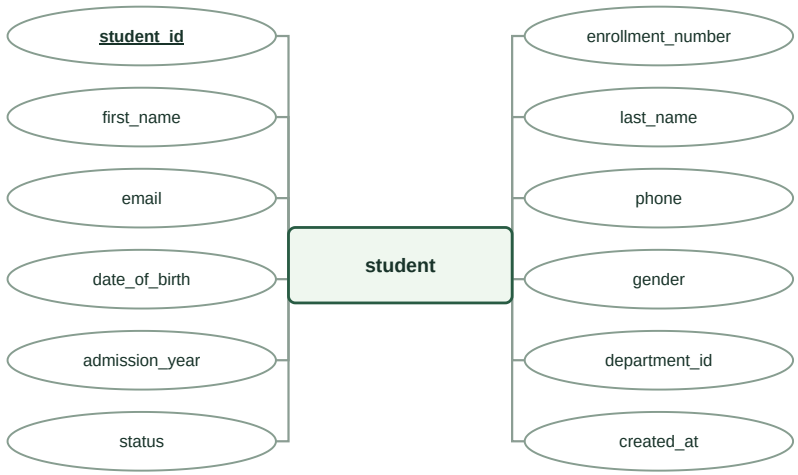
PK: department_id.
UNIQUE: (department_name); (department_code).

Foreign-key references

None. No relationship should be invented merely because fields look similar.

student

Independent student identity and academic membership; no implemented Person superclass.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> Optional department_id allows a student to be temporarily unassigned. Status is Active, Graduated or Inactive. Admission year: 2000–2100. phone stores one scalar value, not a multivalued attribute. Age is not stored.

Attribute	SQL type	Keys	NN	Default
student_id	INTEGER	PK	Yes	—
enrollment_number	VARCHAR(30)	UQ	Yes	—
first_name	VARCHAR(50)	—	Yes	—
last_name	VARCHAR(50)	—	Yes	—
email	VARCHAR(150)	UQ	Yes	—
phone	VARCHAR(15)	—	No	—
date_of_birth	DATE	—	No	—
gender	VARCHAR(20)	—	No	—
admission_year	INTEGER	—	Yes	—
department_id	INTEGER	FK	No	—
status	VARCHAR(20)	—	Yes	'Active'
created_at	TIMESTAMP	—	Yes	CURRENT_TIMESTAMP

Identity & alternate keys

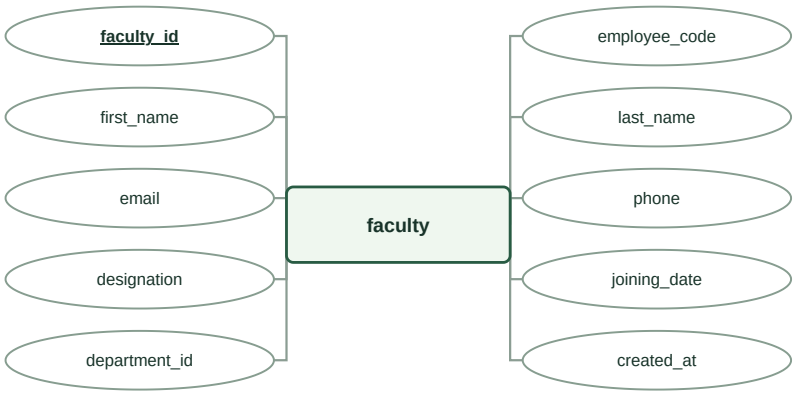
PK: student_id.
UNIQUE: (email); (enrollment_number).

Foreign-key references

`department_id → department.department_id`

faculty

Independent employee identity; departments are mandatory.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> employee_code and email are separately unique. department_id is mandatory. Matching faculty_id and student_id numbers do not identify the same person.

Attribute	SQL type	Keys	NN	Default
faculty_id	INTEGER	PK	Yes	—
employee_code	VARCHAR(30)	UQ	Yes	—
first_name	VARCHAR(50)	—	Yes	—
last_name	VARCHAR(50)	—	Yes	—
email	VARCHAR(150)	UQ	Yes	—
phone	VARCHAR(15)	—	No	—
designation	VARCHAR(100)	—	No	—
joining_date	DATE	—	No	—
department_id	INTEGER	FK	Yes	—
created_at	TIMESTAMP	—	Yes	CURRENT_TIMESTAMP

Identity & alternate keys

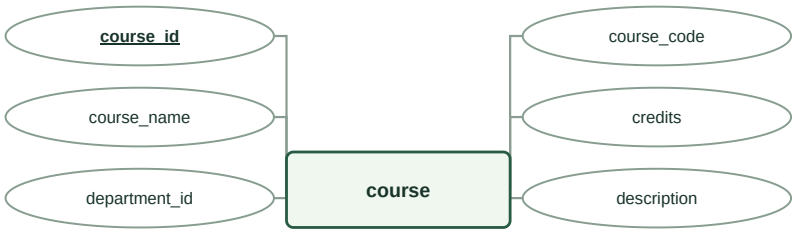
PK: faculty_id.
UNIQUE: (email); (employee_code).

Foreign-key references

`department_id → department.department_id`

course

Catalog definition, not a particular scheduled class.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> Credits: 1–6. Course code follows the implemented letter/digit format. description is optional.

Attribute	SQL type	Keys	NN	Default
course_id	INTEGER	PK	Yes	—
course_code	VARCHAR(20)	UQ	Yes	—
course_name	VARCHAR(150)	—	Yes	—
credits	INTEGER	—	Yes	—
department_id	INTEGER	FK	Yes	—
description	TEXT	—	No	—

Identity & alternate keys

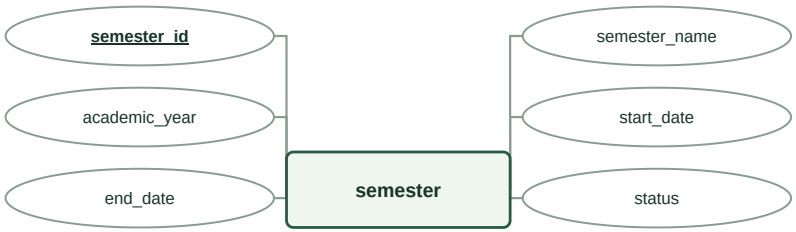
PK: course_id.
UNIQUE: (course_code).

Foreign-key references

`department_id → department.department_id`

semester

A named term within an academic year.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> end_date must be later than start_date. Status: Upcoming, Active or Completed. (semester_name, academic_year) is a composite candidate key.

Attribute	SQL type	Keys	NN	Default
semester_id	INTEGER	PK	Yes	—
semester_name	VARCHAR(50)	—	Yes	—
academic_year	VARCHAR(9)	—	Yes	—
start_date	DATE	—	Yes	—
end_date	DATE	—	Yes	—
status	VARCHAR(20)	—	Yes	'Upcoming'

Identity & alternate keys

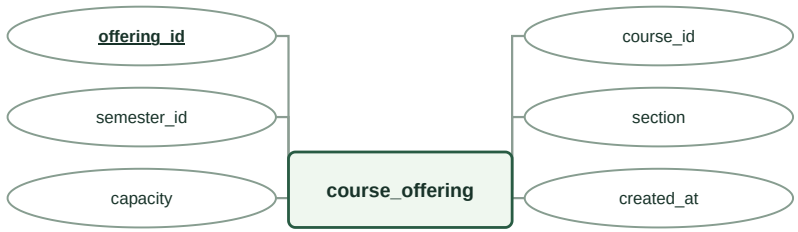
PK: semester_id.
UNIQUE: (semester_name, academic_year).

Foreign-key references

None. No relationship should be invented merely because fields look similar.

course_offering

A course scheduled in a term and section; enrollment capacity belongs here.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> capacity > 0. (course_id, semester_id, section) is unique. Capacity/availability is not encoded by relationship cardinality alone.

Attribute	SQL type	Keys	NN	Default
offering_id	INTEGER	PK	Yes	—
course_id	INTEGER	FK	Yes	—
semester_id	INTEGER	FK	Yes	—
section	VARCHAR(20)	—	Yes	—
capacity	INTEGER	—	Yes	—
created_at	TIMESTAMP	—	Yes	CURRENT_TIMESTAMP

Identity & alternate keys

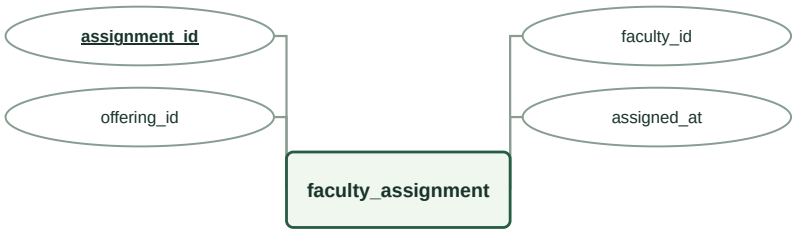
PK: offering_id.
UNIQUE: (course_id, semester_id, section).

Foreign-key references

`semester_id` → `semester.semester_id`
`course_id` → `course.course_id`

faculty_assignment

Associative entity resolving the Faculty–Offering M:N relationship.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> (faculty_id, offering_id) prevents duplicate assignments. Team teaching is structurally possible; no single-teacher limit exists in this pair constraint.

Attribute	SQL type	Keys	NN	Default
assignment_id	INTEGER	PK	Yes	—
faculty_id	INTEGER	FK	Yes	—
offering_id	INTEGER	FK	Yes	—
assigned_at	TIMESTAMP	—	Yes	CURRENT_TIMESTAMP

Identity & alternate keys

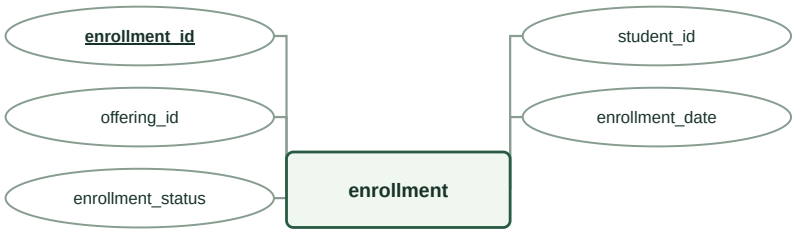
PK: assignment_id.
UNIQUE: (faculty_id, offering_id).

Foreign-key references

`offering_id → course_offering.offering_id`
`faculty_id → faculty.faculty_id`

enrollment

Associative entity resolving Student–Offering M:N; preserves status and registration date.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> (student_id, offering_id) is unique. Status: Enrolled, Completed or Dropped. A dropped row still occupies the unique student/offering pair.

Attribute	SQL type	Keys	NN	Default
enrollment_id	INTEGER	PK	Yes	—
student_id	INTEGER	FK	Yes	—
offering_id	INTEGER	FK	Yes	—
enrollment_date	DATE	—	Yes	CURRENT_DATE
enrollment_status	VARCHAR(20)	—	Yes	'Enrolled'

Identity & alternate keys

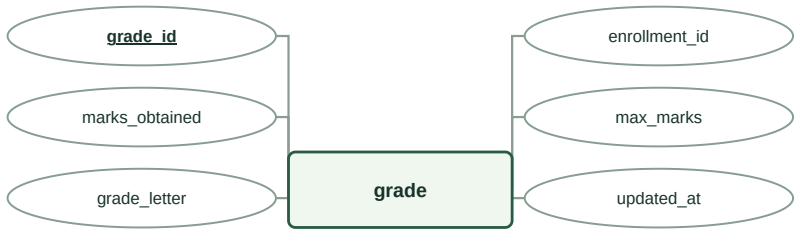
PK: enrollment_id.
UNIQUE: (student_id, offering_id).

Foreign-key references

```
offering_id → course_offering.offering_id
student_id → student.student_id
```

grade

Optional assessment record for one enrollment; a unique FK gives at most one grade.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> max_marks > 0; 0 ≤ marks_obtained ≤ max_marks when non-NULL. grade_letter may be NULL or A+, A, B+, B, C, D, F. grade_letter is stored; the schema does not prove it is automatically derived from marks.

Attribute	SQL type	Keys	NN	Default
grade_id	INTEGER	PK	Yes	—
enrollment_id	INTEGER	FK · UQ	Yes	—
marks_obtained	REAL	—	No	—
max_marks	REAL	—	Yes	—
grade_letter	VARCHAR(2)	—	No	—
updated_at	TIMESTAMP	—	Yes	CURRENT_TIMESTAMP

Identity & alternate keys

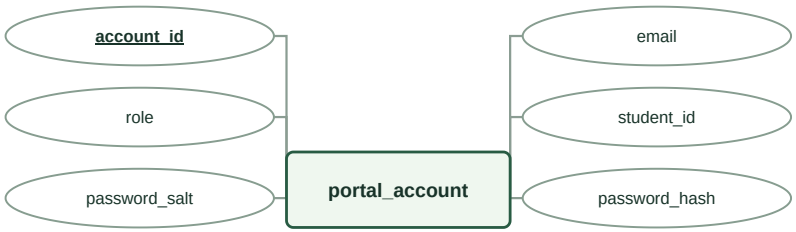
PK: grade_id.
UNIQUE: (enrollment_id).

Foreign-key references

`enrollment_id → enrollment.enrollment_id`

portal_account

Authentication identity; student accounts link to a student, admin accounts do not.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> role ∈ {admin, student}. CHECK: admin ⇒ student_id IS NULL; student ⇒ student_id IS NOT NULL. student_id is unique when present. Account email is NOCASE unique; it is not an FK to student.email.

Attribute	SQL type	Keys	NN	Default
account_id	INTEGER	PK	Yes	—
email	TEXT	UQ	Yes	—
role	TEXT	—	Yes	—
student_id	INTEGER	FK · UQ	No	—
password_salt	TEXT	—	Yes	—
password_hash	TEXT	—	Yes	—

Identity & alternate keys

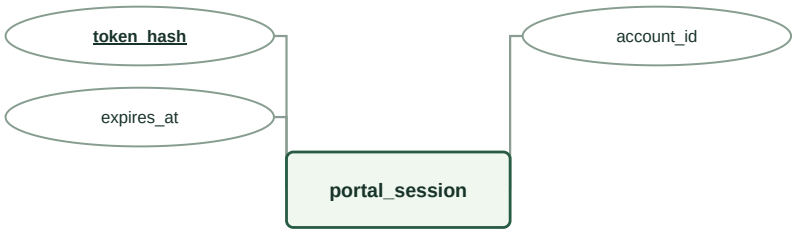
PK: account_id.
UNIQUE: (student_id); (email).

Foreign-key references

`student_id → student.student_id`

portal_session

Expiring authentication session. The stored token hash, not the raw token, is the key.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> account_id is mandatory. Parent account deletion cascades to sessions. expires_at stores an integer time value. Expiry is checked by application logic, not an ER connector.

Attribute	SQL type	Keys	NN	Default
token_hash	TEXT	PK	Yes	—
account_id	INTEGER	FK	Yes	—
expires_at	INTEGER	—	Yes	—

Identity & alternate keys

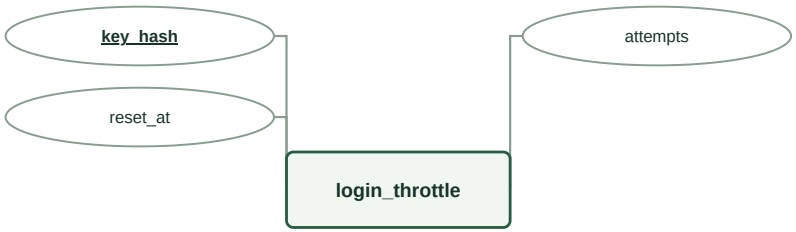
PK: token_hash.
UNIQUE: none beyond the primary key.

Foreign-key references

`account_id → portal_account.account_id`

login_throttle

Operational anti-abuse state keyed by hashed login identity; not a college-domain entity.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> key_hash has no FK to portal_account: login attempts may target nonexistent email identities. attempts and reset_at are operational state.

Attribute	SQL type	Keys	NN	Default
key_hash	TEXT	PK	Yes	—
attempts	INTEGER	—	Yes	—
reset_at	INTEGER	—	Yes	—

Identity & alternate keys

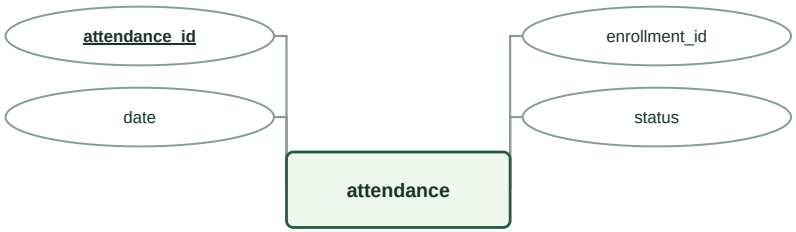
PK: key_hash.
UNIQUE: none beyond the primary key.

Foreign-key references

None. No relationship should be invented merely because fields look similar.

attendance

One status per enrollment and date; records are attached to enrollment, not directly to a student.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> Status: Present, Absent, Late or Excused. (enrollment_id, date) is unique. There is no separate class-session entity in this model.

Attribute	SQL type	Keys	NN	Default
attendance_id	INTEGER	PK	Yes	—
enrollment_id	INTEGER	FK	Yes	—
date	TEXT	—	Yes	—
status	TEXT	—	Yes	—

Identity & alternate keys

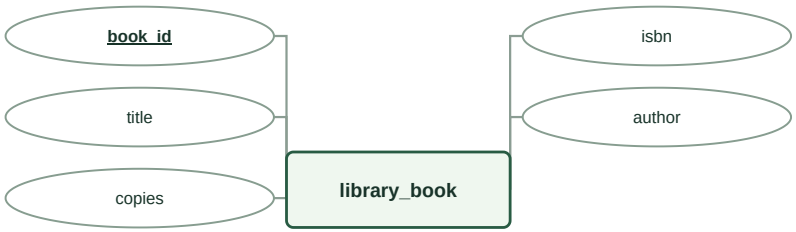
PK: attendance_id.
UNIQUE: (enrollment_id, date).

Foreign-key references

`enrollment_id → enrollment.enrollment_id`

library_book

Bibliographic title with a copies count, not an individually identified physical copy.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> copies > 0; isbn is unique. author is a single TEXT field even when its human-readable value names multiple authors. A normalized multi-author relation would be an extension.

Attribute	SQL type	Keys	NN	Default
book_id	INTEGER	PK	Yes	—
isbn	TEXT	UQ	Yes	—
title	TEXT	—	Yes	—
author	TEXT	—	Yes	—
copies	INTEGER	—	Yes	—

Identity & alternate keys

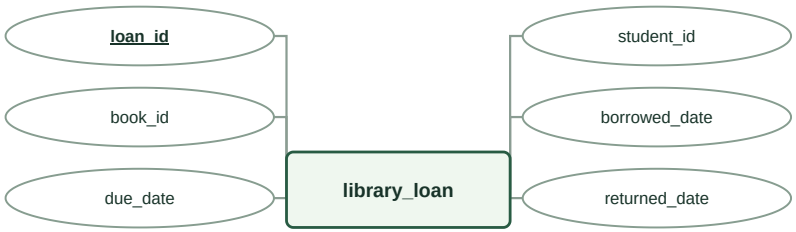
PK: book_id.
UNIQUE: (isbn).

Foreign-key references

None. No relationship should be invented merely because fields look similar.

library_loan

A borrowing event between a student and a book title; repeated borrowing is allowed.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> due_date ≥ borrowed_date; returned_date is NULL or ≥ borrowed_date. No per-copy FK, fine table or stored overdue flag is present.

Attribute	SQL type	Keys	NN	Default
loan_id	INTEGER	PK	Yes	—
student_id	INTEGER	FK	Yes	—
book_id	INTEGER	FK	Yes	—
borrowed_date	TEXT	—	Yes	—
due_date	TEXT	—	Yes	—
returned_date	TEXT	—	No	—

Identity & alternate keys

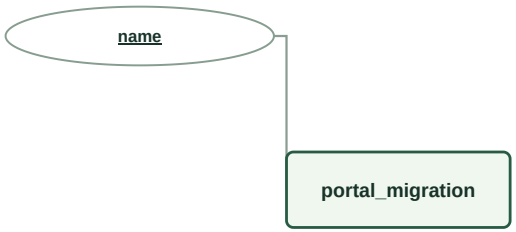
PK: loan_id.
UNIQUE: none beyond the primary key.

Foreign-key references

`book_id` → `library_book.book_id`
`student_id` → `student.student_id`

portal_migration

Operational marker preventing repeat demo-data migrations; not a college-domain entity.



Complete stored-attribute view · underlined = primary key. FK/UNIQUE semantics appear in the adjacent dictionary. All stored columns are single-valued at the relational level.

> Only name is stored. It is a durable initialization marker, not a timestamped audit log.

Attribute	SQL type	Keys	NN	Default
name	TEXT	PK	Yes	—

Identity & alternate keys

PK: name.
UNIQUE: none beyond the primary key.

Foreign-key references

None. No relationship should be invented merely because fields look similar.

Derived values & semantic traps

Value	Derivation / correct interpretation	Status
Full name	first_name + space + last_name. Conceptual Name can be composite; no stored full_name column in student/faculty.	Query/display projection
Age	Completed calendar years between date_of_birth and an as-of date; subtract one if the birthday has not occurred. NULL birth date ⇒ unknown age.	Proposed conceptual derived attribute
Attendance %	100 × Present / all attendance records, rounded. Late and Excused do not count as Present. Zero records must not imply perfect attendance.	Portal computation
Marks %	100 × marks_obtained / max_marks. NULL marks mean not yet available, not zero.	Portal/query computation
Overdue	returned_date IS NULL and due_date earlier than browser's current date. Returned late is a different historical question.	Portal computation; not stored
Enrolled seats	Count enrollments where enrollment_status ≠ Dropped. Completed rows count for the capacity rule.	Capacity/summary computation
Available copies	copies − active unreturned loans for a title. Requires a consistent as-of policy and transactional validation; not a physical-copy identity model.	Derived concept, not a stored column
grade_letter	Although a grading policy could calculate it from marks, the schema stores it independently. Do not draw it as a guaranteed derived attribute.	Stored attribute

> “Derived” is about a computable fact, not whether it is always displayed. A dashed Chen ellipse belongs to a conceptual diagram; a column dictionary must still accurately distinguish actual stored fields.

Read Chen notation before reading the schema

Entity–Relationship modeling describes domain objects, their properties and associations. Enhanced ER (EER) adds concepts such as subtype inheritance. A conceptual diagram explains meaning; a relational schema implements that meaning with tables, keys and constraints. They are related, not interchangeable.

Concept	Standard Chen symbol	Interpretation / example
Strong entity	Single rectangle	Has a complete identifying key of its own: Student, identified by student_id.
Weak entity	Double rectangle	Lacks a complete independent key; identified by owner key plus a partial key.
Relationship	Single diamond	An association between entities, such as Student enrolls in Offering.
Identifying relationship	Double diamond	Connects a classic weak entity to its owner; the weak side participates totally.
Simple / single-valued attribute	Single oval	Simple means not decomposed in this model; single-valued means at most one value per entity. These are different properties.
Composite attribute	Oval branching to component ovals	Name → first_name, last_name. Components carry the stored detail.
Multivalued attribute	Double oval	A set of values for one entity: proposed Student phones, not the actual single phone column.
Stored / derived attribute	Ordinary oval / dashed oval	date_of_birth is stored; age can be calculated from it and an as-of date.
Key / partial key	Solid / dashed underline	A key identifies an entity globally; a weak entity's partial key distinguishes it only within one owner.

Cardinality is not participation

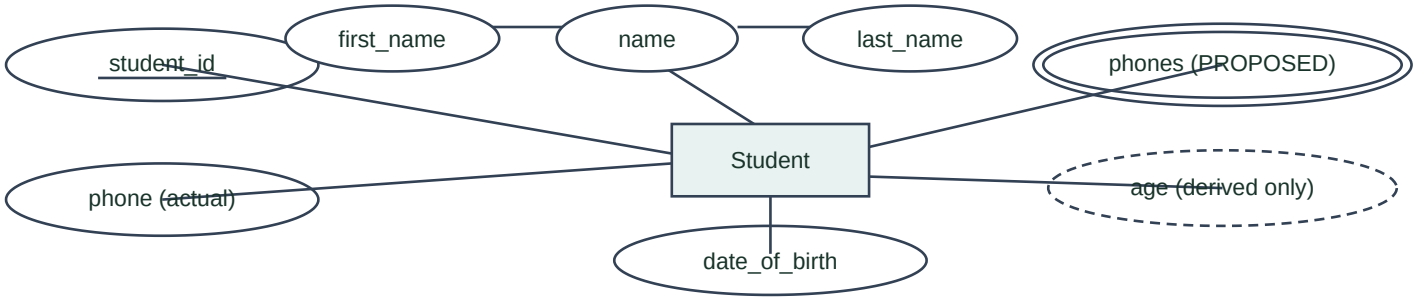
1:1, 1:N and M:N describe maximum association counts. A single connector indicates partial participation; a double connector indicates total participation. Min–max labels such as (0,N) and (1,1) make both bounds explicit. Avoid silently mixing Chen symbols with crow's-foot endpoints.

Keys need business meaning too

Student's student_id is the selected surrogate primary key. In the current schema, enrollment_number and email are also NOT NULL and UNIQUE candidate keys. A composite key contains multiple attributes; a composite attribute contains meaningful components. Neither term implies the other.

Scope and evidence: current facts below refer to the Next.js/SQLite build, particularly web/lib/schema.mjs and the exported CampusOS schema. The independent PostgreSQL build has its own DDL and runtime behavior; it is not the database behind the SQLite build. Proposed diagrams are teaching models, not migrations or claims of deployed features.

Student attributes: structure, multiplicity and derivation



Ordinary ovals = stored components / single values; double oval = proposed set; dashed oval = calculated value. The name oval is conceptual, not an extra stored column.

Composite name, actual single phone

Conceptually, Student.name comprises first_name and last_name. CampusOS already stores these two columns; there is no need to add a redundant name column. “Simple” is relative to modeling needs: first_name is treated as indivisible here even though human naming conventions can be more complex.

The actual student.phone is one nullable VARCHAR(15) value per student. NULL means no recorded value, not an empty collection. A string with commas is still one column value; it does not implement a properly modeled multivalued phone attribute.

Stored input, derived result

date_of_birth is stored and nullable. Age is **not currently stored**; the dashed oval illustrates a possible calculation, not a verified existing feature. Compute completed years as of a stated date, accounting for whether that year’s birthday has occurred. Subtracting birth year alone is inaccurate.

Proposed multivalue → child relation

To support several numbers, introduce `student_phone(student_id, phone)`, with composite PK `(student_id, phone)` and FK `student_id → student(student_id)`. Require both columns; each row holds one number, and the PK prevents duplicate numbers for the same student.

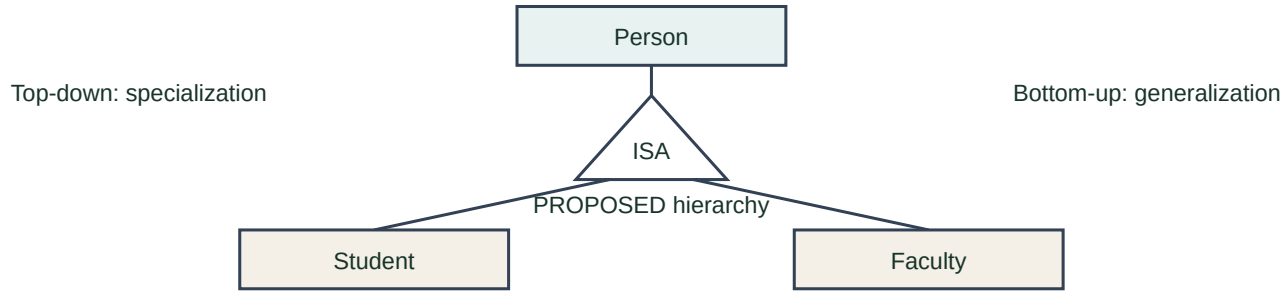
Zero rows represents no recorded phones. Optional metadata such as label or verified_at belongs on the child relation. A “one preferred number” rule needs additional enforcement. If a surrogate phone_id is chosen, retain UNIQUE(student_id, phone).

Migrate existing non-NULL phone values deliberately, adapt forms and queries, then decide whether to remove the old field. Do not keep two competing sources of truth. This child table is **proposed only**.

> **Viva distinction:** composite asks “what parts?”; multivalued asks “how many values?”; derived asks “computed from what?” A name can be composite and single-valued. An unknown birth date should normally yield unknown age; do not invent zero or store a value that becomes stale each birthday.

Specialization ↓ · Generalization ↑

PROPOSED ONLY: CampusOS has separate student and faculty entities with independent primary keys. There is **no Person supertype in the current implementation**. Matching numeric student_id and faculty_id values do not identify the same human.



Chen-style EER extension: rectangles and an ISA triangle. The single supertype connector depicts partial completeness; disjointness/overlap remains a separate design decision.

Specialization: start broad, discover roles

Start with Person and identify more specific entity types: Student has enrollment_number and admission_year; Faculty has employee_code and designation. Shared names and contact facts could move to Person. Each subtype is a subset of Person and inherits its identity, common attributes and applicable relationships.

The subtype is not a second person. In a shared-key mapping, Student.person_id identifies the same entity as Person.person_id. This is an **is-a** relationship, unlike Student belongs to Department, which associates different entities.

Generalization: start specific, discover commonality

Start with Student and Faculty, observe genuinely shared identity and properties, then abstract Person. The resulting diagram may match specialization; the difference is the reasoning direction. Similar columns alone do not justify merging records or prove inheritance.

Unification needs identity matching, conflict resolution and deliberate migration of references. A person can have different institutional email addresses for different roles, so deciding which facts are truly shared is part of the design.

Do not confuse inheritance with authentication

The SQLite portal's admin/student role field governs authorization; it does not establish a Person–Student–Faculty hierarchy. An enrollment associates a student with an offering; it is not a Student subtype. Likewise, Active/Inactive is lifecycle state, not automatically specialization. Use inheritance when shared identity and meaningful subtype-specific rules warrant it—not simply to remove similar-looking column names.

Membership constraints and inheritance mapping

Disjointness controls simultaneous subtype membership. **Completeness** controls whether every supertype entity must be classified. Neither choice follows automatically from the other. All Person examples on this page remain **proposed only**.

Combination	Permitted membership per Person	Possible policy
Total + disjoint	Exactly one of Student, Faculty	Registry includes only these roles and forbids dual membership.
Total + overlapping	At least one; both allowed	Every registered person has an academic role; student-employees are allowed.
Partial + disjoint	Zero or one	Other people may exist, but these two roles are exclusive.
Partial + overlapping	Zero, one or both	Visitors may have neither role; dual-role people may have both.

In a common EER specialization-circle convention, **d** means disjoint and **o** means overlapping; double/single lines from supertype mean total/partial. ISA triangles are another textbook convention. Label the chosen constraints explicitly rather than assuming a triangle alone specifies them.

Three relational strategies

Strategy	Mapping and strengths	Constraints / limitations
Supertype + subtype tables	Person(person_id PK, common facts); Student(person_id PK/FK, student facts); Faculty(person_id PK/FK, faculty facts). Common identity is stored once.	Supports all four combinations with appropriate enforcement. Base PK/FK structure alone permits partial, overlapping membership; shared reads need joins.
One combined table	Person holds common and subtype fields. A NOT NULL discriminator can represent total, disjoint membership.	Conditional CHECKs validate relevant fields. Partial membership needs a “neither” representation; overlap needs multiple flags or memberships, not one exclusive role value.
Concrete subtype tables only	Copy common attributes into Student and Faculty; no Person table. Natural for a total, disjoint hierarchy with independent subtype access.	Partial membership has nowhere to go. Overlap risks duplicated shared facts and needs coordinated identity. Cross-subtype queries need UNION and global-key planning.

What a shared-key FK guarantees

Every subtype row has an existing Person; each subtype PK allows at most one row per Person in that table. It does not require a subtype for every Person and does not prohibit the same ID in both subtype tables.

What needs more enforcement

Total membership and cross-table exclusivity need a suitable schema strategy or carefully designed transactional checks/triggers. SQLite does not provide PostgreSQL-style deferred constraint triggers. Validate rules at a safe transaction boundary; naive immediate checks can reject a valid multi-step insertion.

> **Viva:** “A foreign key points from subtype to supertype. It proves the parent exists, not that every parent has a child. Disjointness is another rule entirely.”

Associative does not mean classically weak

Associative entity: a relationship with facts

Enrollment links Student and CourseOffering and records enrollment_date and enrollment_status. Current CampusOS uses enrollment_id as surrogate PK and UNIQUE(student_id, offering_id) to prevent duplicate registration. FacultyAssignment similarly has assignment_id plus UNIQUE(faculty_id, offering_id).

These are associative entities implementing M:N relationships. Their required foreign keys make them dependent on valid participants, but they are **not classic Chen weak entities** merely because they reference parents. A surrogate key does not remove the need for a business uniqueness rule.

Classic weak entity: owner-relative identity

As a hypothetical example, Dependent might have dependent_name as a partial key within an Employee. Its full relational key is (employee_id, dependent_name). Draw Dependent as a double rectangle, the identifying relationship as a double diamond, and total participation on the weak side.

This example assumes names distinguish dependents within each employee; otherwise choose a better partial key. It is not a current CampusOS table. Existence dependency, mandatory FKs or ON DELETE CASCADE alone do not establish classic weak identification.

Conceptual → relational translation checklist

ER construct	Relational rule	Constraint to retain
Strong entity / attributes	Create a relation; select PK; flatten composite attributes into components. Usually compute derived values rather than storing them.	Other candidate keys remain UNIQUE + NOT NULL where applicable; domains need types and CHECKs.
Multivalued attribute	Create a child relation containing owner FK and one value per row.	Typically PK(owner_key, value); avoid comma lists and repeated phone1/phone2 columns.
Weak entity	Include owner key as FK and part of PK, together with the partial key.	Owner reference is mandatory; deletion policy is a separate business decision.
Binary 1:N	Put the 1-side PK as FK on the N side; place relationship attributes there when appropriate.	NOT NULL enforces mandatory parent per child, not that each parent has children.
Binary 1:1	Use FK + UNIQUE, preferably on the totally participating side; merge only when semantics justify it.	NOT NULL makes that side mandatory. Mandatory participation of the other side needs more enforcement.
M:N / n-ary relationship	Create an association relation with participating FKs and relationship attributes.	Choose key/UNIQUE from cardinality rules; all FKs are not automatically the only candidate key. Binary decompositions may lose n-ary meaning.

Concrete 1:1 check: grade.enrollment_id is NOT NULL, UNIQUE and an FK. Every Grade belongs to exactly one Enrollment; an Enrollment can have zero or one Grade. The schema does not require every Enrollment to have a Grade. Likewise, nullable student.department_id permits an unassigned student; it does not mean an invalid non-NULL department ID is allowed.

Viva questions: definition → evidence → limitation

Examiner asks	Answer to give
Is Student.name implemented as a composite attribute?	Conceptually yes: name decomposes into first_name and last_name. Physically those are separate stored columns; there is no stored name object or required extra full-name column.
Is phone multivalued in your project?	No. Current student.phone stores one optional value. The double-oval phones example is proposed; several numbers would use a child table with student_id FK and uniqueness per student/number.
Where is age stored?	It is not currently stored. date_of_birth is the stored input; age could be derived for an as-of date. Account for birthdays and unknown dates rather than subtracting years blindly.
Does every attribute have just one classification?	No. The dimensions are independent: name can be composite and single-valued; date_of_birth is simple, single-valued and stored. A key is an identification role, not a separate data type.
Specialization versus generalization?	Specialization starts with Person and discovers Student/Faculty top-down. Generalization starts with those types and extracts Person bottom-up. Both can yield the same hierarchy; Person is proposed, not implemented here.
Can a person be Student and Faculty?	A proposed overlapping hierarchy can represent both with one shared identity. Current tables use independent identities; similar names, emails or coincident ID values do not create an explicit identity link.
Does the shared PK/FK enforce the whole hierarchy?	No. It ensures each subtype has a Person and unique membership within that subtype. Disjointness and total completeness require additional enforcement or a different mapping strategy.
Why is Enrollment not a weak entity?	It represents an association, carries relationship facts and has its own surrogate PK. Classic weak identification uses an owner key plus a partial key. Required references alone are not that definition.
Why keep UNIQUE when you already have an ID?	enrollment_id prevents duplicate IDs, not duplicate student/offering pairs. UNIQUE(student_id, offering_id) protects the business identity. The same principle applies to FacultyAssignment.
Does NOT NULL FK mean total participation everywhere?	Only for that referencing side: every child must have a valid parent. It does not require each parent to have a child. Always state the direction of the guarantee.
Is this PostgreSQL running inside Next.js?	No. The Next.js build uses SQLite through Node's node:sqlite. The independent PostgreSQL build has separate DDL and database setup; its locking, typing and trigger mechanisms must not be attributed to SQLite.
Does omitting Person violate normalization?	Not automatically. Normalization concerns functional dependencies within relations. A shared Person identity is an EER/domain design choice, not a mandatory normalization step.

A confident closing answer

"Our implemented model has independent Student and Faculty entities, surrogate-key association tables and database constraints. Composite name explains existing first/last columns; multiple phones and Person ISA are proposed extensions. Age is derived in the conceptual example, not currently stored. I distinguish what the diagram means from what the schema actually enforces."

> **Evidence boundary:** conceptual examples explain alternatives, not completed features. Verify engine-specific SQL against the relevant independent build.